



BARAJA

트럼프 카드 속성 기반 C++ 콘솔 로그라이크 덱빌딩 게임

최종 보고서 | 객체소대 3분대

5764250 최정현 | 5764172 정현우 | 5946130 이승우 | 5820480 서원필

github.com/totalin1324/baraja_cpp

youtube.com/watch?v=KbBHvAemzF0

1. 프로젝트 개요

1.1 작품 주제 선정 배경

최근 게임 시장에서는 반복 플레이를 통해 새로운 경험을 제공하는 로그라이크 장르가 꾸준한 인기를 얻고 있습니다. 특히 Slay the Spire로 대표되는 덱빌딩 로그라이크 장르는 전략적인 카드 조합과 자원 관리가 핵심 재미 요소로 주목받고 있습니다.

본 프로젝트는 이러한 장르의 특징에 트럼프 카드의 4가지 문양(♥ ♦ ♣ ♠)을 속성 시스템으로 채택하고, 포커 족보 메커니즘을 결합하여 단순한 텍스트 게임을 넘어 플레이어의 판단력이 승패를 가르는 완성형 플레이 루프를 구축하는 것을 목표로 하였습니다. CMD 기반의 제한된 환경에서도 색상·ASCII 아트·실시간 상태 표시 등을 활용하여 몰입도를 최대화하였습니다.

1.2 벤치마킹 자료

로그라이크 장르의 원형인 Rogue (1980)를 참고하였습니다. Rogue는 절차적 맵 생성, 턴제 전투, 영구 사망(Permadeath) 시스템을 최초로 도입한 게임으로, 이후 수많은 로그라이크 게임의 기반이 되었습니다. 덱빌딩 시스템은 Slay the Spire의 카드 선택·고정 슬롯 개념을 참고하였습니다.

- Rogue (video game) — Wikipedia: [https://en.wikipedia.org/wiki/Rogue_\(video_game\)](https://en.wikipedia.org/wiki/Rogue_(video_game))
- Binary Space Partitioning — Wikipedia: https://en.wikipedia.org/wiki/Binary_space_partitioning

1.3 개발 환경

구분	내용
HW	Windows 기반 PC 환경
언어	C++17
IDE	Visual Studio Code / Visual Studio 2022
버전 관리	GitHub (totalin1324/baraja_cpp)

라이브러리	기능 및 사용 이유
random (mt19937)	Mersenne Twister 기반 고품질 난수 생성. 맵·카드·몬스터 배치에 사용.
optional	플레이어/몬스터 좌표의 미배치 상태를 null 없이 안전하게 표현.
conio.h (_getch)	Windows 전용 논블로킹 키 입력. Enter 없이 실시간 조작 처리.
thread / chrono	시네마틱 연출·배틀 로그 출력 시 지연(sleep) 처리.

2. 개발 프로그램 설명

2.1 핵심 플레이 루프 (Core Loop)

BARAJA의 게임 흐름은 4단계 순환 구조로 구성됩니다.

단계	내용
01 탐험 (Exploration)	BSP 기반 절차적 랜덤 맵 생성. WASD로 미지의 던전 탐험.
02 조우 & 전투 (Combat)	카드 효과, 포커 족보, 문양 상성을 결합한 전략적 콘솔 전투.
03 보상 (Reward)	전투 승리 후 3장의 무작위 카드 중 선택. 덱 추가 또는 가방 교체.
04 성장 (Progression)	경험치 획득을 통한 스탯 증가 및 고정 슬롯 장착으로 덱 파워 강화.

던전 탐험 : 절차적 맵 생성 (BSP)

BSP Algorithm
이진 공간 분할(Binary Space Partitioning)로
방과 복도를 유기적으로 생성
(RandomWalk 방식과 혼용 가능).

Entity Legend
시안색 '@'는 플레이어, 붉은색 'gG'는 일반/강화
고블린, 노란색 기호는 보스를 직관적으로 표기.

Status Bar
하단에 현재 레벨, HP, 스탯, 고정 카드 슬롯,
덱/버림더미 현황을 실시간 표시.

Legend: @=플레이어 g/G=고블린 K/Q/J/=보스
은스터: 3

▲ 던전 탐험 화면 — BSP 맵, 플레이어(@), 고블린(g), 하단 상태바

2.2 UI/UX 설계

던전 탐험 화면

WASD 키보드로 캐릭터를 이동하며, 이동 목표 칸에 몬스터가 있을 경우 자동으로 배틀 화면으로 전환됩니다. 화면 하단에는 레벨, HP, ATK, DEF, EXP, 고정 슬롯 현황, 덱/버림더미 카드 수가 실시간으로 표시됩니다.

기호	의미
@	플레이어 (시안색)
g / G	일반 / 강화 고블린 (빨간색)
J / Q / K / *	보스 (노란색)
.	바닥 (Floor)
#	벽 (Wall)

배틀 화면 (Battle UI)

몬스터와 인접 시 별도의 배틀 화면으로 자동 전환됩니다.

구성 요소	내용
ASCII Art & HP	색상이 적용된 적/플레이어 ASCII 아트와 [#####---] 형태의 체력바
Hand & Target	[1-5] 숫자 키로 카드 다중 선택. 선택 시 족보 미리보기 및 예상 데미지 제공
Battle Log	카드 효과 발동, 족보 보너스, 상성 데미지(↑/↓)가 컬러 텍스트로 실시간 출력
조작 키	[Enter] 시전, [B] 가방 교체, [S] 건너뛰기

Anatomy of Battle : 텍스트 UI의 극한

18/7

ASCII Art & HP

색상이 적용된 적과 플레이어의 아스키 아트 및 직관적인 [#####---] 형태의 체력바.

Battle Log

카드 효과 발동, 족보 보너스, 상성 데미지 화살표(↑/↓)가 컬러 텍스트로 실시간 출력.

Hand & Target

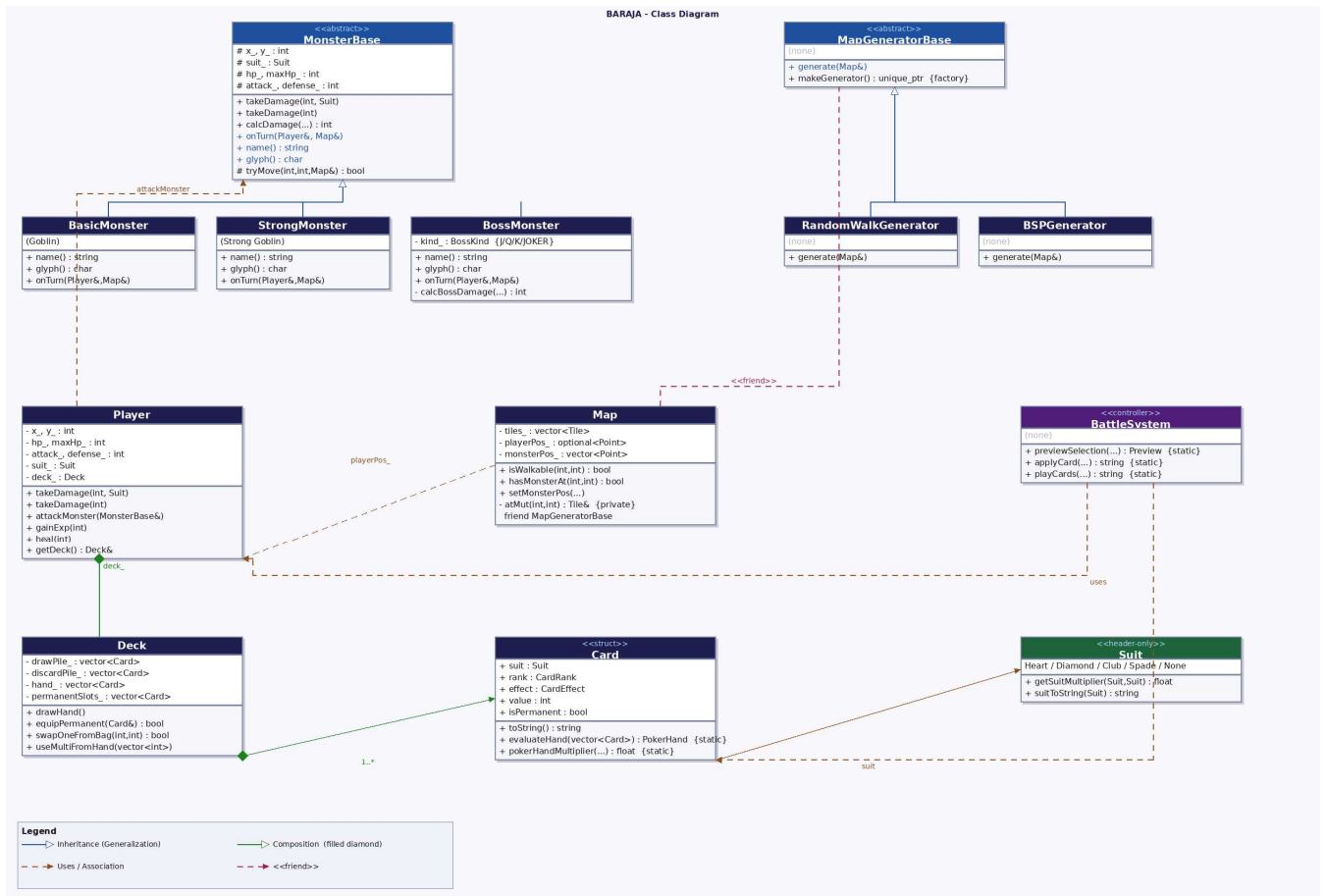
[1-5] 숫자 키로 카드를 다중 선택. 선택 시 노란색 화살표(>)와 족보 미리보기 제공.

[23 - 16905] PREMIUM UNCOATED PAPER |
NotebookLM

▲ 배틀 화면 — ASCII 아트, HP바, 손패 선택, 배틀 로그

2.3 클래스 다이어그램 및 핵심 기능 구현

전체 클래스 다이어그램



전체 구조 개요

```

main
├── Map                                # 타일 데이터 및 엔티티 위치 관리
├── MapGeneratorBase (추상)
│   ├── RandomWalkGenerator          # Drunkard's Walk 동굴형 맵
│   └── BSPGenerator                  # Binary Space Partitioning 방 구조 맵
├── Player                             # 플레이어 상태, 전투, 맥 관리
├── MonsterBase (추상)
│   ├── BasicMonster (Goblin)        # 근접 공격 + 랜덤 이동 AI
│   ├── StrongMonster                # 강화 고블린
│   └── BossMonster                   # J / Q / K / JOKER 고유 패턴 보스
├── BattleSystem                       # 전투 로직 (Static 메서드 컨트롤러)
├── Deck / Card                       # 덱빌딩 - 드로우/핸드/버림/고정 슬롯
└── Suit                              # 카드 속성 상성 시스템 (header-only)
    
```

[Player 클래스]

플레이어 캐릭터의 상태와 행동을 관리하는 클래스입니다. 스탯을 private으로 선언하여 외부에서 직접 수정하지 못하도록 캡슐화하였으며, takeDamage를 두 가지 오버로드로 제공하여 상성 계산이 이중 적용되는 버그를 구조적으로 방지합니다.

```

class Player {
private:
    int hp_, maxHp_, attack_, defense_;
    int level_, exp_, expToNext_;
    Suit suit_;
    Deck deck_; // 텍빌딩 통합 (중간 보고서 대비 추가)
public:
    void takeDamage(int rawDamage, Suit attackerSuit); // 상성·방어력 반영
    void takeDamage(int finalDamage); // 이중 계산 방지용
}
    
```

```
int attackMonster(MonsterBase& target);
void gainExp(int amount);
Deck& getDeck();
};
```

항목	초기값	레벨업 증가
HP	30	+5 (전체 회복)
공격력	8	+2
방어력	3	+1
다음 레벨 경험치	20	×1.5배

[MonsterBase / BasicMonster / StrongMonster / BossMonster 클래스]

MonsterBase는 모든 몬스터의 공통 기능을 담당하는 추상 기반 클래스입니다. 순수 가상 함수로 인터페이스를 강제하고, 각 서브클래스에서 고유 행동 패턴을 구현합니다.

```
class MonsterBase {
public:
    virtual std::string name() const = 0;
    virtual char glyph() const = 0;
    virtual void onTurn(Player& player, Map& map) = 0;
    void takeDamage(int rawDamage, Suit attackerSuit);
    void takeDamage(int finalDamage);
protected:
    bool tryMove(int nx, int ny, Map& map);
};

// BasicMonster: 인접 시 공격, 아닐 시 랜덤 이동
void BasicMonster::onTurn(Player& player, Map& map) {
    if (isAdjacentFour(x_, y_, player.getX(), player.getY()))
        attackPlayer(player);
    else { shuffle dirs; for each dir: tryMove(); }
}
```

클래스	표시	HP	ATK	DEF	EXP	특징
BasicMonster	g	10	4	1	8	랜덤 이동 AI
StrongMonster	G	20	7	3	18	강화 고블린
BossMonster J	J	80	18	9	70	매 2턴 데미지 증가
BossMonster Q	Q	60	15	7	50	HP 절반 시 자가 회복
BossMonster K	K	45	12	5	30	방어 관통 고정 피해
BossMonster JOKER	*	120	22	12	150	4턴 회복 + 광역 패턴

The Bosses : 로그라이크의 최종 관문

18/7

J

J (Jack의 트릭)

연속 공격 패턴 보유.
매 2턴마다 데미지가
추가되는 속전속결형
보스.

Q

Q (Queen의 궁전)

회복 패턴 보유.
체력이 절반 이하일 때
자가 회복하여
장기전을 유도.

K

K (King의 재판)

'왕의 일격'.
매 3턴마다 방어력을
관통하는 강력한 단일
고정 피해를 입힘.

*

★ JOKER (최종 보스)

4턴 주기로 회복,
우랑타, 이중타, 옴사의
난동(광역/대폭발)을
난사하는 최종 시련.

[23 - 16905] PREMIUM UNCOATED PAPER |

NotebookLM

▲ 보스 4종 — J (Jack의 트릭), Q (Queen의 궁전), K (King의 재판), JOKER

[Map / MapGeneratorBase 클래스]

Map은 게임 맵의 전체 타일 데이터를 관리하는 컨테이너 클래스입니다. 타일 쓰기 메서드 atMut()를 private으로 선언하고 MapGeneratorBase를 friend로 지정하여, 맵 생성기 계층에서만 타일 수정이 가능하도록 캡슐화하였습니다.

```
// 타일 쓰기를 생성기 계층에만 허용 (캡슐화)
friend class MapGeneratorBase;
Tile& atMut(int x, int y); // private

// 팩토리 함수: 호출 측이 구체 타입에 의존하지 않도록 추상화
auto gen = Roguelike::makeGenerator(Roguelike::GeneratorType::BSP, rng);
gen->generate(map);
```

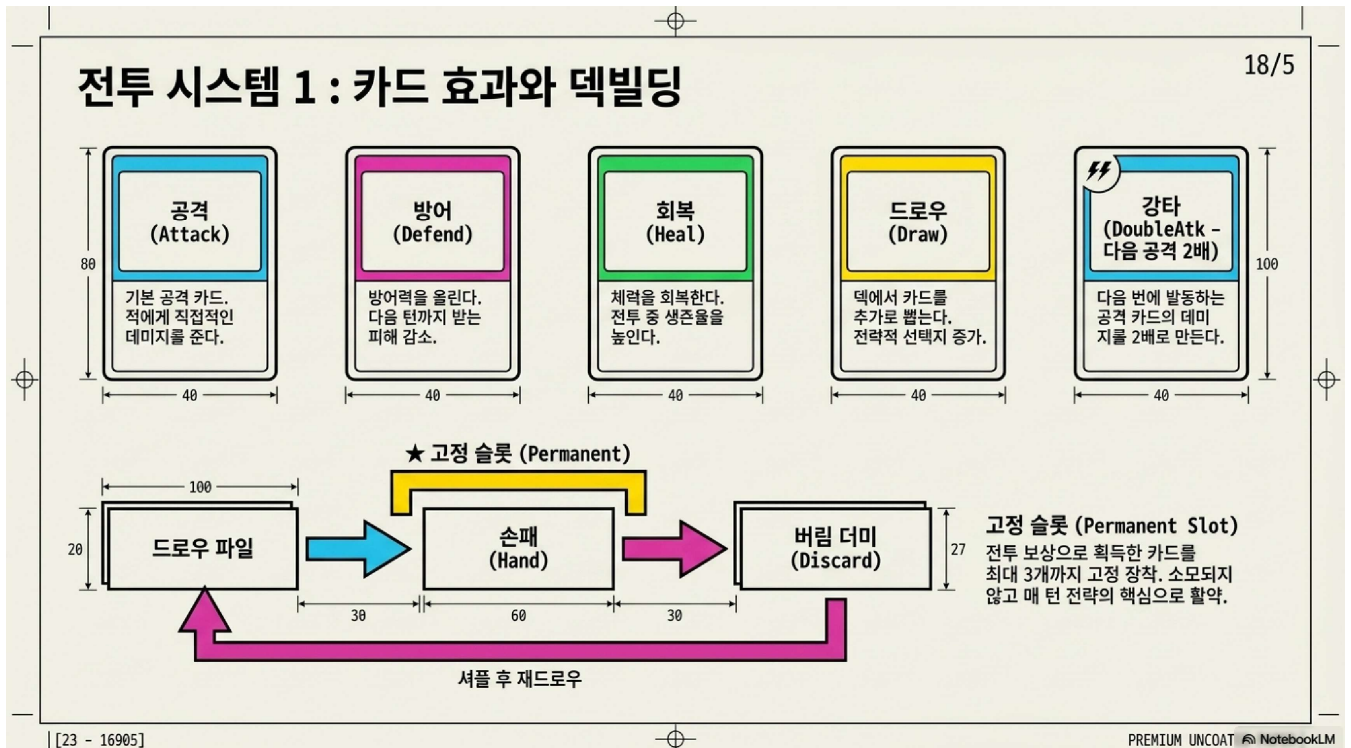
생성기	알고리즘	특징
RandomWalkGenerator	Drunkard's Walk	유기적인 동굴형 맵
BSPGenerator	Binary Space Partitioning	방과 L자형 복도 구조 맵

[Card / Deck 클래스]

중간 보고서 이후 새로 구현된 덱빌딩 시스템의 핵심입니다. 카드의 속성(Suit), 숫자(CardRank), 효과(CardEffect), 값(value), 고정 여부(isPermanent)를 하나의 구조체로 관리합니다.

카드 효과	내용
Attack	기본 공격 카드. 상성 배율 + 포커 족보 보너스 적용.
Defend	이번 턴 방어력을 일시적으로 올린다.
Heal	플레이어 HP를 회복한다.

카드 효과	내용
DrawCard	덱에서 카드를 추가로 드로우한다.
DoubleAtk	다음 공격 카드의 데미지를 2배로 만든다.



▲ 카드 5종 및 드로우 파일 → 손패 → 버림 더미 순환 구조

```
class Deck {
    enum { HAND_SIZE = 5, MAX_PERM = 3 };
    // 드로우 파일 -> 손패(Hand) -> 버림 더미 순환
    // 고정 슬롯(permanentSlots_): 소모되지 않고 매 턴 손패에 포함
    bool equipPermanent(const Card& card);
    bool swapOneFromBag(int handIdx, int bagIdx);
    void swapHandFromBag(const vector<int>& bagIndices);
private:
    vector<Card> drawPile_, discardPile_, hand_, permanentSlots_;
};
```

[BattleSystem 클래스]

전투 로직 전체를 담당하는 컨트롤러 클래스입니다. 상태를 정적(Static) 메서드로 처리하여 데이터 오염을 방지합니다.

```
class BattleSystem {
public:
    static SelectionPreview previewSelection(
        const vector<Card>& played, const Player&, const MonsterBase&);
    static string applyCard(const Card&, Player&, MonsterBase&, BattleState&);
    static string playCards(
        const vector<int>& indices, Player&, MonsterBase&, Deck&, string& pokerResult);
};
```

[Suit 시스템]

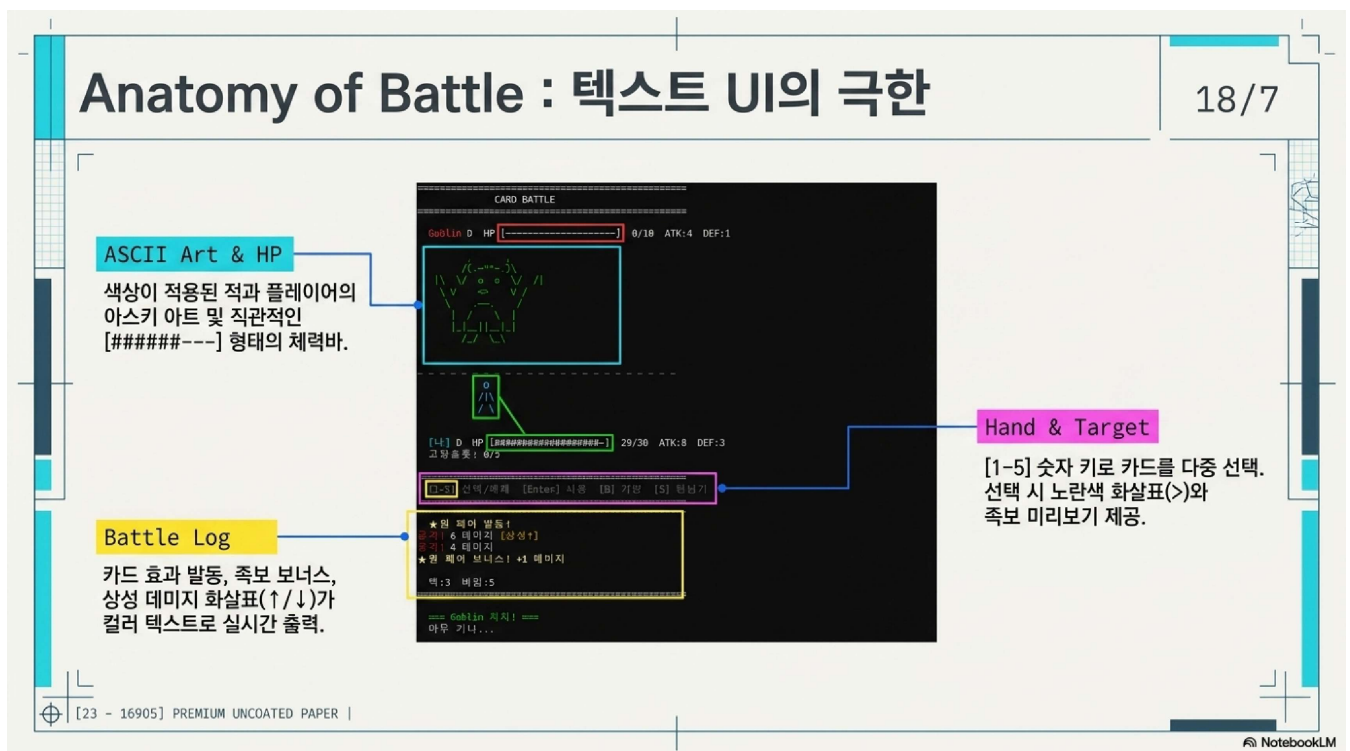
트럼프 카드의 4가지 문양을 enum class Suit : uint8_t로 정의하고, 상성 관계를 getSuitMultiplier(attacker, defender) 단일 함수로 집중 관리합니다. header-only 구현으로 어디서든 포함 가능합니다.

상성 순환: Diamond > Heart > Spade > Club > Diamond

관계	배율
유리 (상성)	1.5×
불리 (역상성)	0.75×
중립 / 무속성 (None)	1.0×

[포커 족보 시스템]

배틀 화면에서 1~5장의 카드를 선택하면 카드의 문양(Suit)과 숫자(CardRank)를 기반으로 포커 족보를 평가하여 공격 데미지에 배율 보너스를 적용합니다.



▲ 카드 속성 상성(좌) 및 포커 족보별 데미지 배율(우)

족보	배율	족보	배율
원 페어	1.2×	풀 하우스	2.3×
투 페어	1.4×	포 카드	2.8×
트리플	1.6×	스트레이트 플러시	3.5×
스트레이트	1.8×	로얄 플러시	5.0×
플러시	2.0×	없음	1.0×

[스테이지 시스템]

총 10개의 스테이지로 구성되며, 스테이지별 몬스터 구성과 보스 유형이 고정 배열(STAGES[10])로 관리됩니다.

스테이지	이름	일반	강화	보스
1	던전 입구	3	0	-
2	깊어지는 위험	0	4	-
3	Jack의 트릭	2	0	J (Jack)
4	침울한 판교	0	4	-
5	적의 영역	0	5	-
6	Queen의 궁전	2	0	Q (Queen)
7	불타는 검은 숲	0	4	-
8	저운 침묵	0	5	-
9	King의 재판	2	0	K (King)
10	JOKER의 재판	0	0	* (JOKER)

2.4 객체지향 요소 정리

OOP 요소	적용 위치	설명
추상 클래스	MonsterBase, MapGeneratorBase	순수 가상 함수로 서브클래스의 구현 강제
상속	BasicMonster/StrongMonster/BossMonster → MonsterBase	공통 기능을 기반 클래스에서 상속받아 재사용
오버라이딩	onTurn(), name(), glyph(), generate()	서브클래스에서 기반 클래스의 가상 함수를 재정의
오버로딩	takeDamage() — Player, MonsterBase	동일 함수명으로 raw/final 두 버전 제공
캡슐화	Player, Map, Deck	멤버를 private 선언, getter/setter로만 접근
다형성	MonsterBase* → BasicMonster 등	unique_ptr<MonsterBase>로 가상 함수 동적 디스패치
friend	Map ↔ MapGeneratorBase	타일 쓰기 권한을 생성기 계층에만 제한적으로 허용
Static 메서드	BattleSystem	상태 오염 없는 순수 전투 로직 컨트롤러 구현
헤더/소스 분리	Player, Monster, Map, MapGenerator	선언(.h)과 구현(.cpp)을 파일로 분리

3. 중간 보고서 대비 추가 구현 사항

항목	중간 보고서 상태	최종 구현 결과	상태
몬스터 이동 AI	미구현 (인접 공격만)	BasicMonster/StrongMonster: 인접 시 공격, 아닐 시 방향 셔플 후 이동. tryMove()로 충돌 판정 통합.	완료
배틀 UI (별도 화면)	맵 화면에서 인라인 처리	몬스터 인접 시 전용 배틀 화면 자동 전환. ASCII 아트, HP바, 손패, 족보 미리보기, 배틀 로그 실시간 출력.	완료
스테이지 진행 시스템	단일 맵, 종료 구조	10스테이지 고정 배열(STAGES[10]). 클리어 시 보상 후 다음 스테이지. 보스 스테이지(3-6-9-10)에 컷신 연출.	완료
덱빌딩/카드 시스템	미정	Card/Deck/BattleSystem 신규 구현. 드로우→손패→버림 순환, 고정 슬롯(최대 3), 가방 교체, 전투 보상 3선택 완성.	완료
포커 족보 시스템	미정 (속성 상성만)	카드 문양·숫자 기반 원 페어~로얄 플러시 9종 평가. 족보 배율로 보너스 데미지 적용.	완료
보스 몬스터	미구현	BossMonster에 J/Q/K/JOKER 4종 구현. 고유 패턴 로직 적용.	완료
엔딩 시네마틱	미구현	JOKER 처치 시 JOKER DEFEATED 배너 → 탈출 → 야외 → 크레딧 ASCII 연출.	완료

4. 테스트 및 검증 / 장애요인과 문제 해결

4.1 테스트 및 검증

테스트 항목	내용	결과
전투 상성 계산	♥ 플레이어가 ♠ 고블린 공격 시 1.5x 배율 적용 확인	정상
포커 족보 평가	동일 숫자 2장 → 원 페어(1.2x), 5장 동일 문양 → 플러시(2.0x) 확인	정상
덱 순환	드로우 파일 소진 시 버림 더미 셔플 후 재드로우 처리 확인	정상
고정 슬롯	고정 카드 매 턴 손패 포함, 사용 후 소모되지 않음 확인	정상
레벨업	경험치 누적 후 다중 레벨 동시 상승 처리 확인	정상
맵 생성	BSP/RandomWalk 알고리즘으로 매 실행마다 다른 맵 생성 확인	정상
몬스터 AI	인접 시 공격, 비인접 시 랜덤 이동, 벽/몬스터 충돌 차단 확인	정상
보스 패턴	J 2턴 데미지 증가, Q HP 절반 자가 회복, K 방어 관통 확인	정상
스테이지 진행	10스테이지 순차 진행, 보스 스테이지 컷신 전환 확인	정상
승리/패배 조건	JOKER 처치 시 엔딩, HP 0 시 게임 오버 확인	정상

4.2 개발 중 장애요인과 문제 해결

장애요인	문제 해결
순환 참조: Player.h와 Monster.h가 서로를 include하면 컴파일 오류	전방 선언(forward declaration)으로 순환 참조 차단
takeDamage 이중 계산: 상성 계산 후 raw takeDamage를 재호출하면 배율 두 번 적용	takeDamage를 오버로드하여 최종 데미지를 그대로 적용하는 버전 별도 제공
중첩 struct 기본 인수 오류: Config 클래스 내부 선언 시 CWG 325 제약으로 컴파일 오류	Config 구조체를 클래스 외부(네임스페이스 스코프)로 분리
엔티티 위치 중복 배치: 플레이어와 몬스터가 같은 좌표에 배치되는 경우 발생	uniqueFloor() 함수로 이미 사용된 좌표를 추적하여 겹침 방지
미리보기와 실제 적용 공식 불일치로 예상/실제 데미지 차이 발생	previewSelection()과 playCards()의 공식을 동일하게 유지하도록 리팩터링
가방 교체 시 인덱스 밀림: 낮은 인덱스부터 제거하면 뒤 인덱스가 밀려 오작동	bagIndices를 내림차순 정렬 후 제거 처리 (swapHandFromBag 구현)
LNK4221 / 중복 심볼 경고: 헬퍼 함수가 여러 파일에서 정의됨	Color, initConsole, clearScreen 등을 main.cpp 안에서만 static으로 정의

5. 현재까지의 성과 및 결론

5.1 기대효과 및 느낀점

본 프로젝트는 로그라이크 장르와 덱빌딩·포커 족보 RPG 요소를 결합하여 높은 반복 플레이성과 전략적인 전투 경험을 제공합니다. 속성 상성과 포커 족보가 중첩 적용되는 구조는 단순 공격 반복에서 벗어나 상황 판단과 카드 조합 전략을 핵심 재미 요소로 만듭니다. CMD 기반으로 가벼운 환경에서도 실행 가능하며, OOP·자료구조·알고리즘을 실습하기 위한 교육용 프로젝트로도 활용할 수 있습니다. 향후 카드 희귀도 (Rarity) 시스템, 세이브/로드 기능, 전투 로그 리플레이 시스템을 추가하면 상용 인디 게임 수준으로 확장 가능합니다.

처음에는 단순한 로그라이크를 만들 생각이었는데, 덱빌딩과 포커 족보까지 합쳐지면서 생각보다 훨씬 복잡한 시스템이 되었습니다. Map과 MapGenerator를 추상화 계층으로 분리하는 설계가 나중에 스테이지 시스템을 붙일 때 얼마나 유용한지 직접 체감했습니다. 처음부터 OOP 원칙을 지키면서 설계하면 나중에 기능 확장이 얼마나 편해지는지 이번 프로젝트를 통해 깊이 배웠습니다.

5.2 프로젝트 성과

항목	상태
카드 속성 상성 시스템	완료
포커 족보 평가 시스템	완료
덱빌딩 (드로우 / 핸드 / 버림 / 고정 슬롯)	완료
전투 보상 카드 선택 시스템	완료
타일 맵 컨테이너	완료
맵 자동 생성 (RandomWalk, BSP)	완료
플레이어 / 몬스터 전투 시스템	완료
레벨업 시스템	완료
몬스터 이동 AI	완료
배틀 UI (별도 화면 전환)	완료
스테이지 진행 시스템 (10단계)	완료
보스 몬스터 (J / Q / K / JOKER)	완료
보스 컷신 및 엔딩 시네마틱	완료
기본 게임 루프 / main.cpp	완료
카드 희귀도 (Rarity) 시스템	미구현 (Future)
세이브 / 로드 기능	미구현 (Future)

6. 개발 일정 및 팀원 업무 분장

6.1 개발 일정

기간	내용
5 / 6	프로젝트 시작 / GitHub 리포지토리 생성 / 헤더 파일 초안 작성
5 / 7	Map-MapGenerator 구현 / Player-Monster 구현 / 속성 상성 시스템 / 게임 루프 작성
5 / 8	버그 수정 / README 작성 / 중간 보고서 제출
5월 중순	덱빌딩 시스템 (Card / Deck / BattleSystem) 구현
5월 중순	배틀 UI 별도 화면 구현 / 포커 족보 평가 시스템 구현
5월 하순	몬스터 이동 AI / 보스 몬스터(J/Q/K/JOKER) / 스테이지 시스템
6월 초	엔딩 시네마틱 / 전체 통합 테스트 / 버그 수정 / 최종 보고서 작성
6 / 7	최종 제출

6.2 팀원 업무 분장

이름	학번	역할
최정현	5764250	전체 기획 및 설계 / Map-MapGenerator 구현 / 게임 루프(main.cpp) / 스테이지 시스템 / 몬스터 이동 AI
이승우	5946130	Player-Monster 구현 / Suit 속성 상성 / Card-Deck 클래스 / 포커 족보 평가 시스템 / 전투 보상 카드 시스템
정현우	5764172	배틀 UI 구현 (별도 화면 전환) / BattleSystem 컨트롤러 / ASCII 아트 연출 / previewSelection 구현 / 보스 컷신 구현 / 엔딩 시네마틱
서원필	5820480	보스 몬스터(J/Q/K/JOKER) 구현 / 보스 고유 패턴 설계 / 스테이지별 밸런스 조정

7. 참고문헌

- [1] Rogue (video game) — Wikipedia: [https://en.wikipedia.org/wiki/Rogue_\(video_game\)](https://en.wikipedia.org/wiki/Rogue_(video_game))
- [2] Binary Space Partitioning — Wikipedia: https://en.wikipedia.org/wiki/Binary_space_partitioning
- [3] Slay the Spire — MegaCrit: <https://www.megacrit.com/>
- [4] C++ Reference: <https://en.cppreference.com/>